

Review 1: Quality of the Module Developed

First of all, an essential aspect of the developed module is the presence of a structure in the system and the successful use of several advanced technologies within the model. It should be mentioned that the process of creating the model was performed on the basis of a rather rich COVID-19 Radiography Database, which includes many samples of chest radiograph images. The interaction with such a resource allowed considering important aspects, such as class distributions, data preparation, and training processes, contributing greatly to the creation of the proposed system.

Another crucial feature of the module is high computational efficiency due to the possibility to work with GPU. Training models in Google Colab using GPU resources allowed performing faster tests and avoiding problems related to the necessity to process big amounts of data and train complicated structures of deep learning models.

Moreover, it is important to note that another aspect of the quality of the module concerns its architecture. In the context of the described project, one can observe how the system successfully combines such important components as data preprocessing, model prediction, explaining AI, and report generation. Moreover, each component performs its function while improving the performance of the overall system.

In addition to the previous aspects, it is vital to note the presence of the feature of the generated explanations by the system. The ability to see visual explanations of results and use the information provided by AI about the disease increases the quality of the tool significantly.

Overall, it should be noted that the created project demonstrates the high quality of its modules since theoretical ideas are combined with the implementation of the solution..

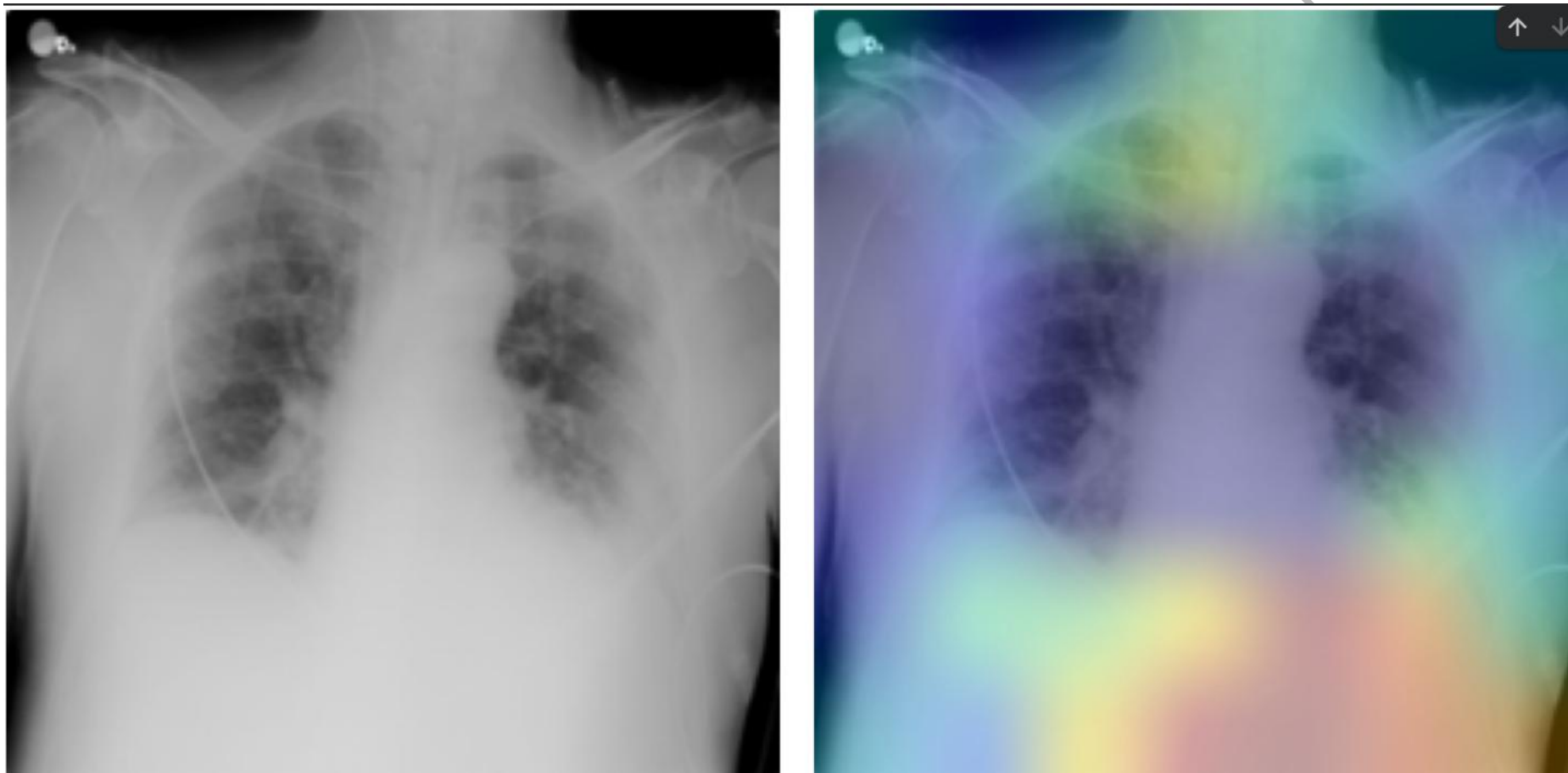


Figure 69: Untrained model false detection before fine tuning

AFFEEF AHMAD

Step 1. Original X-ray



Step 2. Grad-CAM novelty



Figure 70:: Huge detection improvement after fine-tuning the model

AFEEF AHMED

Review 2: Containerization and Deployment

One of the most important features of the developed COVID-19 Detection System is the presence of clear mechanisms for containerization and deployment which make the system easily reproducible, portable and fit for productive use.

During the process of system creation, a special focus was put on following modern software architecture and deployment trends. Therefore, containerization was used in accordance with the modern principles based on Docker and Docker Compose technologies.

System Architecture:

The developed COVID-19 Detection System can be viewed as a multi-component architecture including the following parts:

- **Frontend:** React app that uses Vite for development and optimized building
- **Backend:** FastAPI server for images preprocessing, orchestration and API management
- **Model Inference:** NVIDIA Triton Inference Server for efficient, high-performance inference of models
- **Local Model:** As a fallback, Keras model can be used to ensure more reliable results

All of these components were packaged separately into containers by means of Docker technology. It guarantees that no external dependencies exist, and all necessary libraries are packed.

Docker Compose Orchestration

In order to facilitate the launch of multiple services, Docker Compose technology was used. The file `docker-compose.yml` describes services, their configuration, dependencies and networking. The system can be started with the following command:

```
docker compose up -d
```

This command does the following:

- Building and launching of Triton Inference Server with COVID classifier model loaded
- Starting a FastAPI server with correct health check setup and dependencies
- Launching React application
- Setting up inter-service communication and volumes for model loading

NVIDIA Triton Model Serving

When deploying a system, production-ready NVIDIA Triton Inference Server is used for efficient inference of pre-trained models. For setting up the service, the corresponding Docker image from NVIDIA repository is downloaded first:

```
docker pull nvcr.io/nvidia/tritonserver:23.10-py3
```

Service Endpoints

After deployment, all service endpoints can be accessed using the following URLs:

- **Frontend:** <http://localhost:3000>— React Vite application for uploading X-rays and visualizing results
- **Backend API:** <http://localhost:8010/docs> — FastAPI documentation with prediction endpoint
- **Triton Server:** <http://localhost:8001>— NVIDIA Triton Inference Server for models inference (HTTP), <http://localhost:8002> (gRPC)

Benefits of Deployment

The following are the key benefits of this deployment solution:

- **Consistency:** Standardized execution in development, testing, and production stages
- **Mobility:** All services can be executed on any device that supports Docker and Docker Compose
- **Scalability:** Individual scaling of services based on demand (front-end, back-end, inference)
- **Resilience:** Health checks guarantee the availability of services; fault tolerance is provided by the mixed inference pipeline using Triton and local Keras
- **Deployment Ready:** Enterprise-level orchestration that is capable of being deployed to cloud-based systems or on-site infrastructure

Medical AI Diagnostic Portal

Agentic AI integration for hospital resource optimization and triage support.

Analyses
3

Status
Ready

Size
0.04 MB



Change Image

Run AI Prediction

Copy Summary

Clear Session

Diagnosis Results

High confidence

Classification:

COVID

AI Confidence:

91.98%

What To Do Next

Likely Condition: COVID-19 pattern suspected

Urgency: Prompt physician review recommended within the same day.

Recommended Actions

- Isolate patient as per local infection prevention protocol.
- Check oxygen saturation, respiratory rate, and temperature at triage.
- Correlate with symptoms and confirm using RT-PCR/rapid antigen as indicated.

Red Flags

- SpO2 below 94%
- Increasing shortness of breath
- Persistent high fever or chest pain

Follow-Up: Reassess clinically in 12 to 24 hours or earlier if symptoms worsen.

Figure 71: Main Dashboard

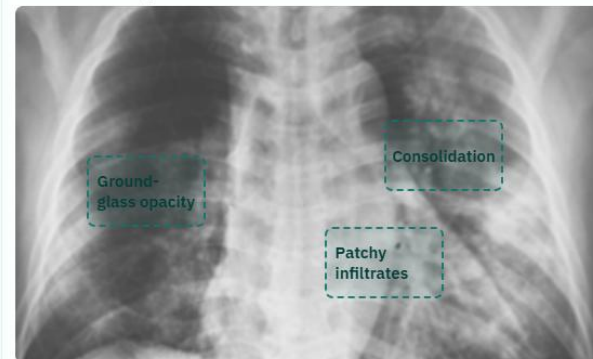
Medical AI Diagnostic Portal	123 Trichy, India
Doctor Name	Specialization
Dr. Olivia Greene	Doctor
Visit Date	Patient Name
05/03/2026 	Sarah Anderson
Birth Date	Medical Number
01/01/1989 	MA567891
IHI	Phone
5556-9669-9654-7788	+1 (555) 789-0123
Email	Hospital Contact
s.anderson@mail.com	+91 (XXX) 123-4567
Hospital Email	Hospital Site
info@Medical_AI_Diagnostic.com	www.Medical_AI_Diagnostic.com

Source: remote-llm | Latency: 9296 ms | Gemini-backed LLM

Impression: Multifocal, predominantly peripheral ground-glass opacities and consolidations are present bilaterally, significantly more pronounced in the left lung. These findings are highly suggestive of viral pneumonia, such as COVID-19, given the clinical context and high AI confidence. Correlation with RT-PCR testing and clinical symptoms is strongly recommended.

Narrative Summary: The frontal chest radiograph demonstrates extensive, patchy opacities distributed throughout both lung fields. There is a notable peripheral and mid-to-lower zone predominance, which is characteristic of atypical or viral pneumonias. The left lung shows more confluent areas of consolidation compared to the right, particularly in the mid-lung zone. No significant pleural effusions or pneumothorax are identified. The cardiomedastinal silhouette remains within normal limits for size and configuration, and the visualized osseous structures are unremarkable.

LLM Pattern Map



- **Ground-glass opacity:** Patchy peripheral opacities in the right mid-to-lower lung zone.

Figure 72: LLM model Insights

LLM Annotated Image

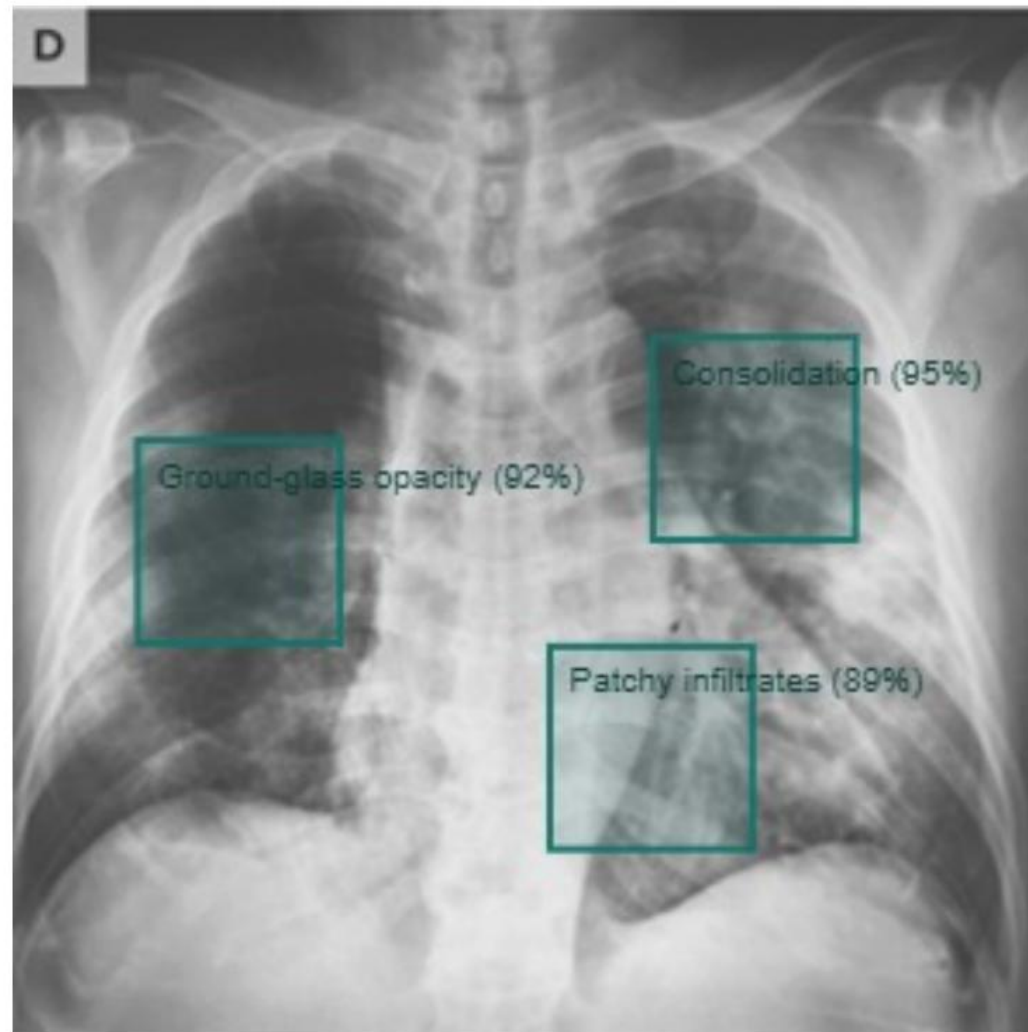


Figure 73: Model Overlay of Detected virus

PERSONAL

Search

Ctrl+K

?

4

A

-

×

Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Upgrade plan

seaaai

D:\seaaai

View configurations

api	INFO:	127.0.0.1:40050	- "GET /health HTTP/1.1" 200 OK
afeefahmed/seaaai:8010:8010	api-client	INFO:	127.0.0.1:52096 - "GET /health HTTP/1.1" 200 OK
		INFO:	127.0.0.1:53560 - "GET /health HTTP/1.1" 200 OK
triton-server	api	INFO:	127.0.0.1:40076 - "GET /health HTTP/1.1" 200 OK
seaaai-triton-server:8000:8000	api-client	INFO:	127.0.0.1:56450 - "GET /health HTTP/1.1" 200 OK
Show all ports (3)	api	INFO:	127.0.0.1:50136 - "GET /health HTTP/1.1" 200 OK
api-client	api-client	INFO:	127.0.0.1:49298 - "GET /health HTTP/1.1" 200 OK
seaaai-api-client:8011:8001	api	INFO:	127.0.0.1:53052 - "GET /health HTTP/1.1" 200 OK
frontend	api-client	INFO:	127.0.0.1:44514 - "GET /health HTTP/1.1" 200 OK
seaaai-frontend:3000:80		INFO:	127.0.0.1:43372 - "GET /health HTTP/1.1" 200 OK
	api	INFO:	127.0.0.1:43338 - "GET /health HTTP/1.1" 200 OK
	api-client	INFO:	127.0.0.1:36108 - "GET /health HTTP/1.1" 200 OK
	api	INFO:	127.0.0.1:49668 - "GET /health HTTP/1.1" 200 OK
	api-client	INFO:	127.0.0.1:37116 - "GET /health HTTP/1.1" 200 OK
	api	INFO:	127.0.0.1:34674 - "GET /health HTTP/1.1" 200 OK
	api-client	INFO:	127.0.0.1:59560 - "GET /health HTTP/1.1" 200 OK
		INFO:	127.0.0.1:60654 - "GET /health HTTP/1.1" 200 OK
	api	INFO:	127.0.0.1:48212 - "GET /health HTTP/1.1" 200 OK
	api-client	INFO:	127.0.0.1:40018 - "GET /health HTTP/1.1" 200 OK
	api	INFO:	127.0.0.1:59322 - "GET /health HTTP/1.1" 200 OK

Engine running

RAM 3.46 GB CPU 0.37% Disk: 66.77 GB used (limit 1006.85 GB)

Terminal v4.71.0

Figure 74: All containers running in docker

125